



Laboratory Report

Experiment 3: Automatic Classification and Sorting of Desktop Objects

Part 1: Visual Recognition

Course	Robot Integration Group Project
Name	Wenshuo Liu
Student ID	24020036044
Role	Part 1: Visual Recognition
Responsibility	Camera + YOLO + ROS 2 Detection Node
Group	Exp_3_group_5
Training Platform	NVIDIA GeForce RTX 4060
Deployment Platform	NVIDIA Jetson Orin NX
Date	September 2026

Real-Time Visual Perception for Desktop Object Sorting with Three-Stage Domain Adaptation

Olivia

College of Information Science and Engineering
Ocean University of China
Group 5 – Exp_3_group_5

Abstract—This report presents the complete development of the visual perception subsystem for a desktop object sorting task. The subsystem detects two object categories, Square (rectangular chewing-gum boxes) and Circle (tape rolls), and publishes their class IDs, bounding boxes, and confidence scores through ROS 2 `vision_msgs/Detection2DArray` messages.

Rather than using a single-stage training workflow, the detector was progressively developed through three successive stages. First, a YOLOv8n model was trained using 676 manually collected real-object images. Second, simulation images obtained from the Gazebo environment were annotated and incorporated into a simulation-domain fine-tuning stage. Third, after deployment to the physical robotic platform, 48 additional images captured directly from the real robot camera were added for further transfer learning.

The initial detector achieved an overall mAP50 of 0.991. During deployment, several engineering problems were also identified and solved, including class-ID inconsistency, NumPy and ROS 2 `cv_bridge` compatibility, camera overexposure, and a significant viewpoint-induced domain gap affecting the Circle class.

After real-camera fine-tuning, both object classes could be detected in the physical sorting environment. The final ROS 2 detector operated at approximately 30 Hz on a Jetson Orin NX.

Index Terms—YOLOv8, ROS 2, Jetson Orin NX, object detection, robotic perception, domain adaptation, simulation-to-real.

I. INTRODUCTION

Experiment 3 aims to construct a complete robotic sorting pipeline consisting of perception, decision making, grasping, classification, placement, and exception handling.

The overall project was divided into four functional parts. My responsibility was Part 1, the visual perception subsystem, including camera acquisition, dataset construction, YOLO-based object detection, model training and fine-tuning, Jetson deployment, and ROS 2 detection-message publication.

The sorting task contains six target objects: three Square objects and three Circle objects. The Square class corresponds to rectangular chewing-gum boxes, whereas the Circle class corresponds to tape rolls.

The visual subsystem is required not only to display detection results but also to provide standardized, machine-readable perception information for the downstream grid-localization and robot-control modules.

The complete development process contained three model training stages:

- 1) initial training using the manually collected object dataset;
- 2) simulation-domain fine-tuning using images obtained from the Gazebo environment;
- 3) physical-camera fine-tuning using images captured directly from the real robot platform.

This progressive training strategy was necessary because the visual appearance of the same objects changed substan-



Fig. 1. Physical Square and Circle objects used in the sorting task.

tially between the original data-collection environment, the simulation environment, and the final physical robotic system.

II. SYSTEM ARCHITECTURE

A. Perception Pipeline

The complete visual processing pipeline is

$$\begin{aligned} \text{Camera} &\rightarrow \text{ROS 2 Image} \rightarrow \text{YOLOv8} \\ &\rightarrow \text{Class Mapping} \rightarrow \text{Detection2DArray}. \end{aligned} \quad (1)$$

The detector subscribes to the image topic

`/image_raw`

and publishes all detected objects through

`/detections`.

B. Object Definition

The two classes used in the experiment are shown in Fig. 1.

Additional examples from the original data-collection environment are shown in Fig. 2.

C. Bounding-Box Representation

For a YOLO detection represented by the bounding-box coordinates



Fig. 2. Examples of the Circle and Square objects in the original data-collection environment.

$$(x_1, y_1, x_2, y_2),$$

the center coordinates are calculated as

$$x_c = \frac{x_1 + x_2}{2}, \quad y_c = \frac{y_1 + y_2}{2}. \quad (2)$$

The bounding-box width and height are

$$w = x_2 - x_1, \quad h = y_2 - y_1. \quad (3)$$

These quantities are written into the `Detection2D.bbox` field before publication.

D. Class-ID Mapping

An important interface inconsistency was identified after the initial detector had already been trained.

The training dataset used

$$0 = \text{Circle}, \quad 1 = \text{Square},$$

whereas the team-level interface convention required

$$0 = \text{Square}, \quad 1 = \text{Circle}.$$

Since the detector itself had already achieved high recognition accuracy, relabeling the complete dataset and retraining the model would have introduced unnecessary work without improving the learned visual representation.

Instead, a deterministic class-ID conversion was implemented inside the ROS 2 detector immediately before message publication.

This approach separates the model's internal label definition from the external team interface and guarantees that the downstream modules always receive the agreed class IDs.

III. STAGE 1: INITIAL DATASET TRAINING

A. Data Collection

The first stage used real photographs captured with a mobile phone from approximately top-down viewpoints.

A total of 676 original images were collected.

The objects were manually annotated using LabelMe. After annotation, the dataset contained

- 869 Circle bounding boxes;
- 866 Square bounding boxes.

The labeled data were converted to the Ultralytics YOLO format.

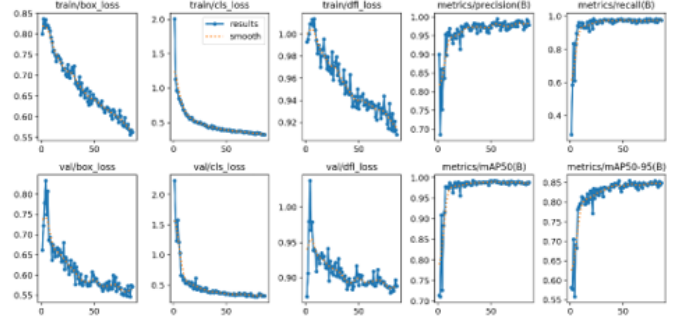


Fig. 3. Stage 1 training curves for the initial 676-image dataset.

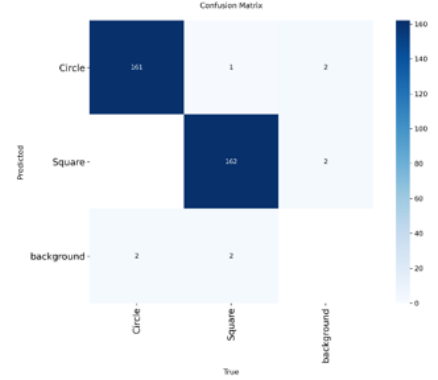


Fig. 4. Confusion matrix obtained after Stage 1 training.

The final dataset division was

- 474 training images;
- 135 validation images;
- 67 test images.

B. Training Configuration

YOLOv8n was selected as the detection network because the final model was required to run in real time on an embedded Jetson platform.

The Stage 1 training configuration was

- input resolution: 640;
- batch size: 16;
- optimizer: AdamW;
- maximum number of epochs: 150;
- early-stopping patience: 30;
- training GPU: NVIDIA RTX 4060 with 8 GB memory.

Although the maximum number of epochs was set to 150, training stopped at epoch 86 because the early-stopping condition was reached.

C. Initial Training Results

The training curves are shown in Fig. 3.

The confusion matrix obtained after the initial training stage is shown in Fig. 4.

Example validation predictions are shown in Fig. 5.

The initial detector achieved an overall mAP50 of 0.991 and an mAP50-95 of 0.855.

For Circle, the initial results were

$$P = 1.000, \quad R = 0.975,$$

$$\text{mAP50} = 0.994, \quad \text{mAP50-95} = 0.838.$$

For Square, the initial results were

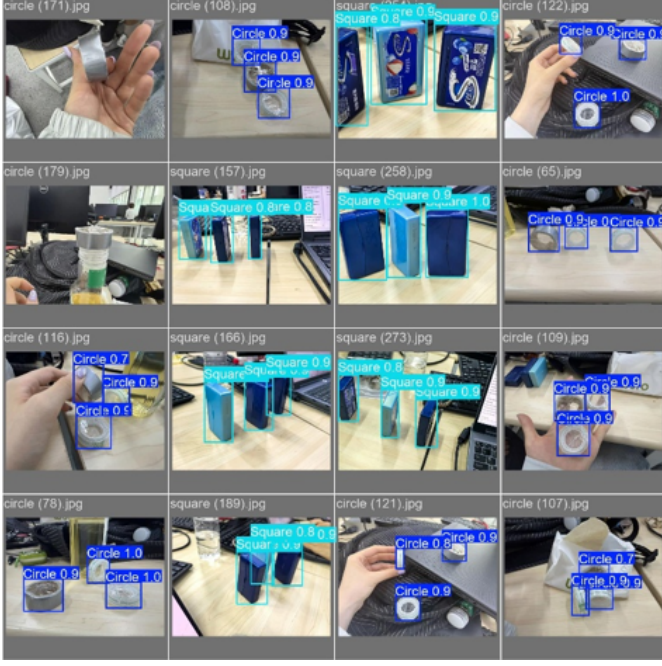


Fig. 5. Validation predictions produced by the Stage 1 detector, including scenes containing both classes.

$$P = 0.991, \quad R = 0.982,$$

$$\text{mAP50} = 0.988, \quad \text{mAP50-95} = 0.872.$$

These results demonstrated that the initial dataset provided a strong representation of the two objects under the original data-collection conditions.

However, subsequent simulation and physical experiments showed that high accuracy on this distribution did not guarantee equally strong performance after the camera domain changed.

IV. STAGE 2: SIMULATION-DOMAIN FINE-TUNING

A. Motivation for Simulation Adaptation

After Stage 1, the trained detector was integrated into the Gazebo-based robot simulation.

This introduced a second visual domain.

The simulation images differed from the original mobile-phone images in several aspects, including object rendering, background appearance, lighting, viewpoint, object placement, and camera geometry.

Therefore, the simulation images were not treated merely as visualization screenshots.

Instead, they were explicitly incorporated into a second training stage so that the model could adapt to the simulation environment before physical deployment.

B. Semi-Automatic Simulation Annotation

To prepare the simulation data, an annotation utility was implemented using OpenCV and the previously trained Ultralytics model.

The Stage 1 detector was first loaded and applied to the simulation image using a confidence threshold of 0.50.

At this point, the Square objects could already be detected with relatively high confidence.

Therefore, the annotation script automatically selected the three strongest Square detections.

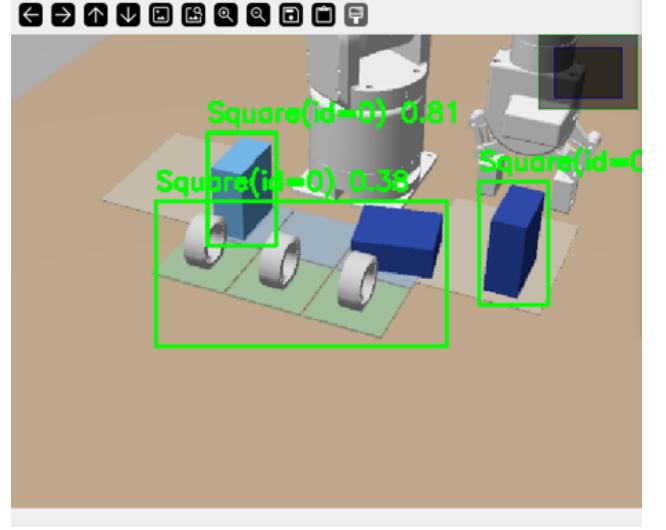


Fig. 6. Simulation scene before domain adaptation. Square objects can already be detected, whereas Circle objects are not yet reliably represented.

The Circle objects were then manually annotated.

For each Circle object, the upper-left corner and the lower-right corner of the desired bounding box were selected using mouse clicks.

The annotation interface also provided a reset operation. If an incorrect point was selected, all clicks could be cleared and the annotation process restarted.

Only after all three Circle bounding boxes had been completed were the labels written to disk.

C. YOLO Label Conversion

For an image of width W and height H , each manually selected rectangle was converted to normalized YOLO coordinates.

The normalized center coordinates were

$$x_{c,n} = \frac{x_1 + x_2}{2W}, \quad (4)$$

$$y_{c,n} = \frac{y_1 + y_2}{2H}, \quad (5)$$

while the normalized width and height were

$$w_n = \frac{x_2 - x_1}{W}, \quad (6)$$

$$h_n = \frac{y_2 - y_1}{H}. \quad (7)$$

The training labels retained the original model convention

$$0 = \text{Circle}, \quad 1 = \text{Square}.$$

The conversion to the group-level convention remained a separate operation inside the ROS 2 detector.

D. Simulation Before Fine-Tuning

The simulation scene before domain adaptation is shown in Fig. 6.

A mixed prediction montage used to inspect the detector across different image domains is shown in Fig. 7.

E. Simulation Fine-Tuning

After annotation, the simulation-domain images were incorporated into the training data.

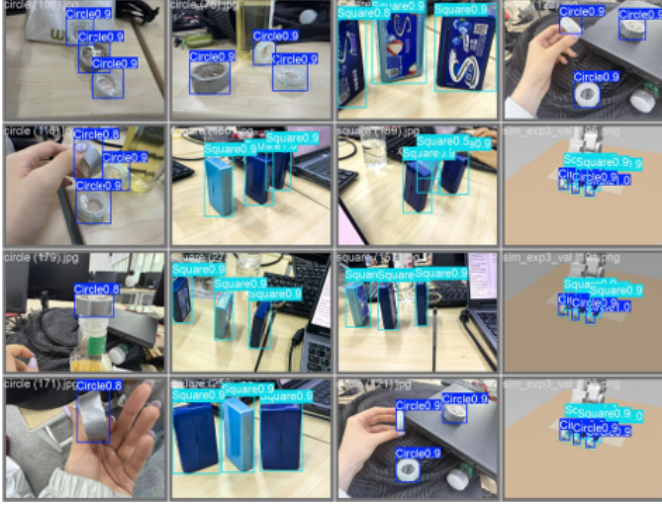


Fig. 7. Prediction montage containing both original-domain and simulation-domain images.

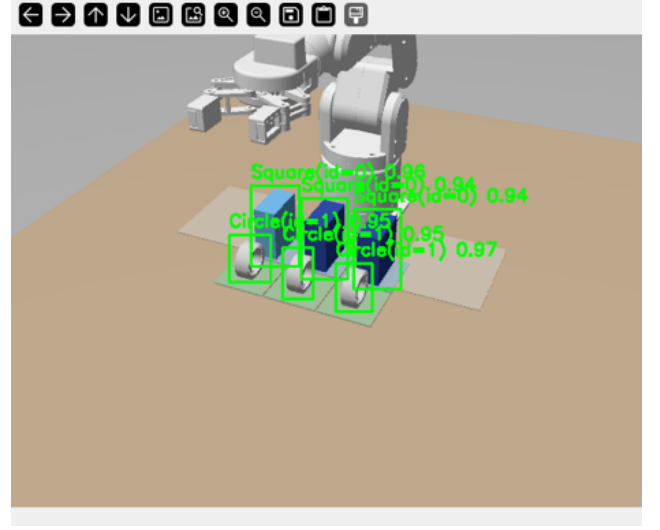


Fig. 9. Detection result after Stage 2 simulation-domain adaptation. All six tabletop objects are detected in the Gazebo environment.

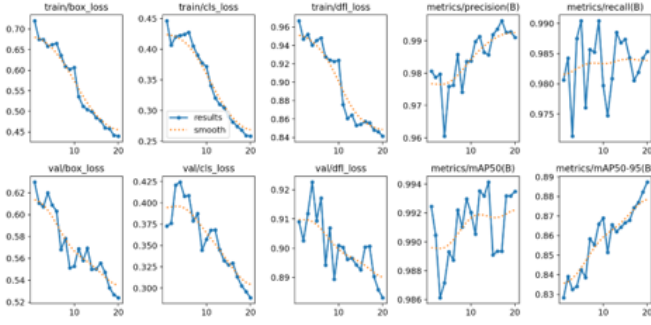


Fig. 8. Training curves obtained during simulation-domain fine-tuning.

Rather than initializing a new network from random weights, the Stage 1 `best.pt` model was used as the starting point for local fine-tuning.

This transfer-learning strategy was intended to preserve the strong object features already learned from the original photographs while exposing the detector to the appearance and geometry of the Gazebo environment.

The Stage 2 training curves are shown in Fig. 8.

F. Simulation Verification

After Stage 2 fine-tuning, the detector was tested again in the Gazebo environment.

The result is shown in Fig. 9.

The detector was able to recognize all six tabletop objects, including three Square objects and three Circle objects.

This stage is important to the experimental chronology.

The development process did not jump directly from the initial phone-image model to real-robot fine-tuning.

Instead, the actual sequence was

Initial Training → Simulation Fine-Tuning → Physical Fine-Tuning (8)

V. JETSON ORIN NX AND ROS 2 DEPLOYMENT

A. Embedded Deployment Environment

After simulation verification, the trained weights were transferred to a Jetson Orin NX running ROS 2 Humble.

The deployment environment included

- Ultralytics YOLO;

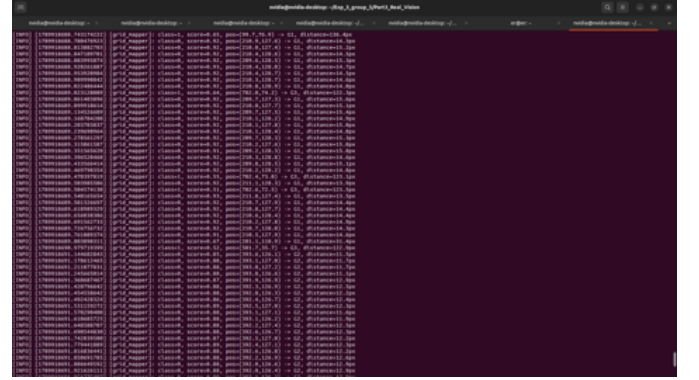


Fig. 10. ROS 2 detector runtime output on the Jetson platform.

- OpenCV;
- `cv_bridge`;
- `usb_cam`;
- `vision_msgs`;
- the custom `experiment3_vision` ROS 2 package.

The detector node performs image subscription, OpenCV conversion, YOLO inference, class-ID mapping, message construction, ROS 2 publication, and optional real-time visualization.

B. ROS 2 Detection Node

The detector performs the following operations for every incoming camera frame:

- 1) subscribe to `/image_raw`;
- 2) convert the ROS image into an OpenCV image using `cv_bridge`;
- 3) execute YOLO inference;
- 4) extract bounding-box coordinates and confidence scores;
- 5) convert the internal model class ID to the team-standard class ID;
- 6) construct a `vision_msgs/Detection2DArray`;
- 7) publish the result through `/detections`.

A runtime example is shown in Fig. 10.

C. NumPy and cv_bridge Compatibility

A major deployment problem occurred after installing Ultralytics using `pip`.

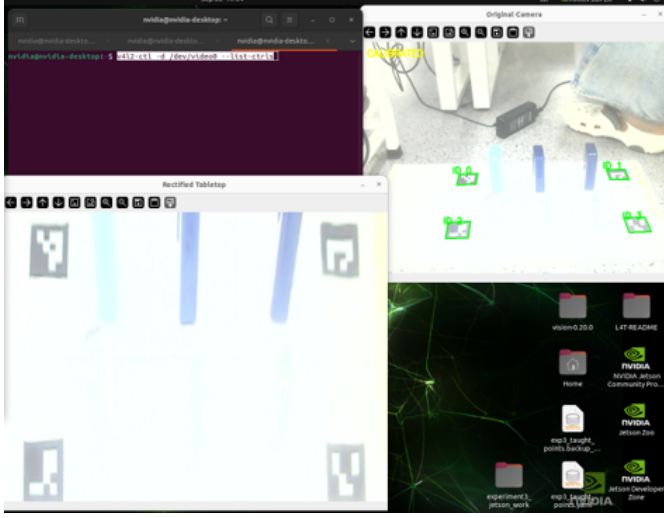


Fig. 11. Integrated ROS 2 debugging environment during the camera-exposure investigation.

The installation upgraded NumPy to the 2.x series.

However, the ROS 2 Humble Python bindings available on the Jetson, including `cv_bridge`, had been compiled against NumPy 1.x.

This ABI mismatch produced the error

`_ARRAY_API not found.`

The solution was to downgrade and pin NumPy to the 1.x series.

After correcting the NumPy version, `cv_bridge` and OpenCV were tested independently before the complete detector node was launched again.

This step was important because it separated a software environment problem from potential errors in YOLO inference or ROS 2 communication.

D. Camera Exposure Problem

During integrated testing, the USB camera initially produced a severely overexposed image.

The debugging interface is shown in Fig. 11.

Investigation showed that the camera brightness parameter had been set to the abnormal value 10, whereas the expected value was 0.

The parameter was reset using `v4l2-ctl`.

After correction, the camera produced a usable tabletop image, as shown in Fig. 12.

After the software and camera problems had been resolved, the detector operated at approximately 30 Hz.

This frequency was close to the camera acquisition rate and was sufficient for the real-time requirements of the sorting task.

VI. STAGE 3: PHYSICAL-ROBOT FINE-TUNING

A. Initial Physical-Robot Test

After successful simulation integration, the detector was deployed on the physical robotic platform.

During the first physical test, the Square class remained detectable with confidence values of approximately 0.84–0.85.

However, the Circle objects were almost completely missed.

The failure is illustrated in Fig. 13.

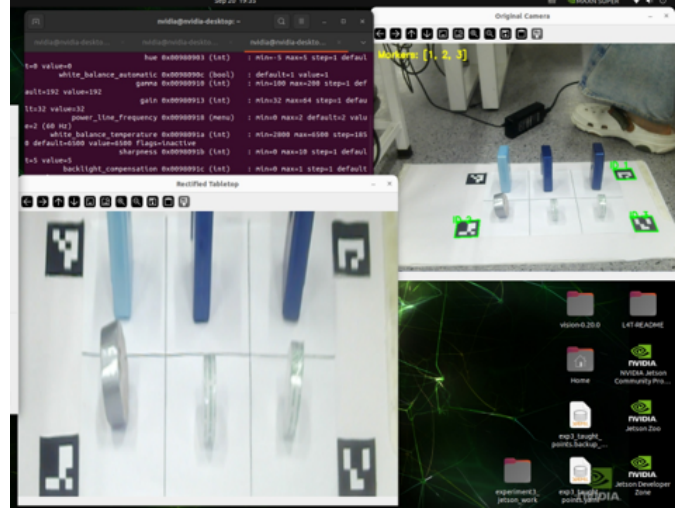


Fig. 12. Camera image after correcting the exposure-related parameter.

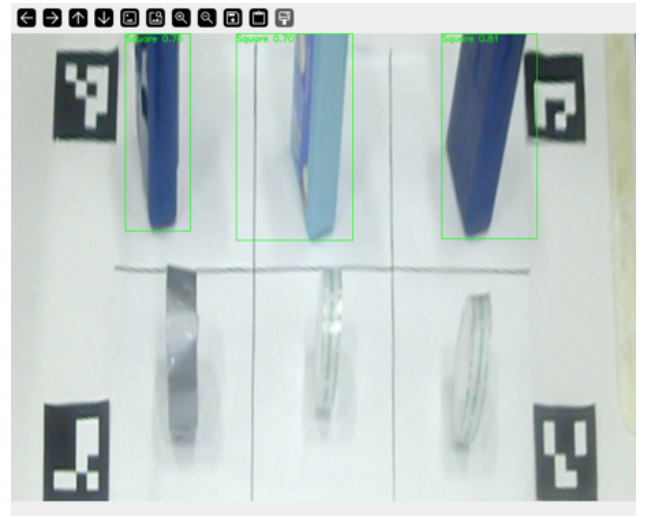


Fig. 13. Initial physical-robot test. Square objects are detected, whereas the Circle objects are missed.

B. Domain-Gap Diagnosis

The confidence threshold was first reduced to determine whether the Circle objects were simply receiving low confidence values.

This modification did not solve the problem.

Therefore, the failure was unlikely to be caused only by the threshold.

A visual comparison between the original training images and the physical-camera images revealed a substantial viewpoint-induced domain gap.

In the original mobile-phone dataset, the tape rolls typically appeared as relatively complete circular rings.

In the physical robot setup, however, the camera observed the tabletop from a lower elevation angle.

The stronger perspective distortion compressed the visible shape of the tape rolls into narrow arcs.

The Square objects were considerably less sensitive to this viewpoint change.

This explains why Square recognition remained relatively stable while Circle recognition degraded significantly.

C. Real-Camera Data Collection

After identifying the viewpoint mismatch, an OpenCV image-acquisition utility was used to collect additional train-

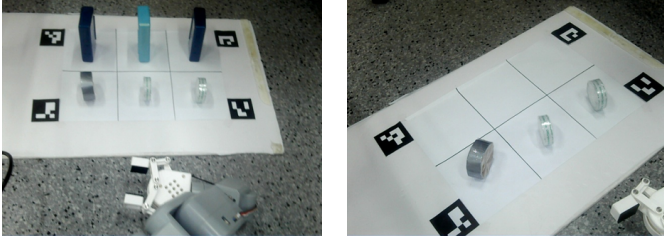


Fig. 14. Additional images collected directly from the physical robot camera. The oblique viewpoint substantially changes the apparent geometry of the Circle objects.

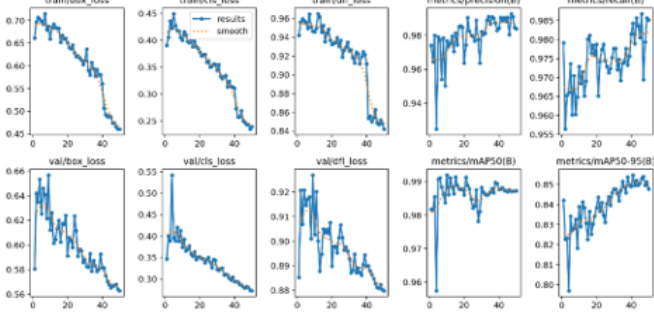


Fig. 15. Stage 3 training curves after introducing the physical-camera samples.

ing images directly from the camera installed on the physical manipulator.

A total of 48 additional images were collected.

Unlike the original phone dataset, these images were captured under the actual deployment conditions, including

- the real camera viewpoint;
- the real object-to-camera distance;
- the physical tabletop background;
- the actual lighting conditions;
- the manipulator and gripper geometry visible in the scene.

Both Square and Circle objects were annotated whenever they appeared in an image.

This was necessary because leaving visible objects unlabeled could incorrectly teach the network that those objects were part of the background.

Examples of the newly collected physical-camera images are shown in Fig. 14.

D. Final Transfer Fine-Tuning

The 48 newly annotated images were divided into

- 40 training images;
- 8 validation images.

These images were then merged with the existing dataset. The previous model weights were used as the initialization point.

The detector was fine-tuned for 50 additional epochs instead of being trained from scratch.

This transfer-learning strategy was intended to preserve the previously learned object representation while adapting the network to the actual physical camera geometry.

The Stage 3 training curves are shown in Fig. 15.

E. Final Physical Test

After fine-tuning, the model was deployed to the physical system again.

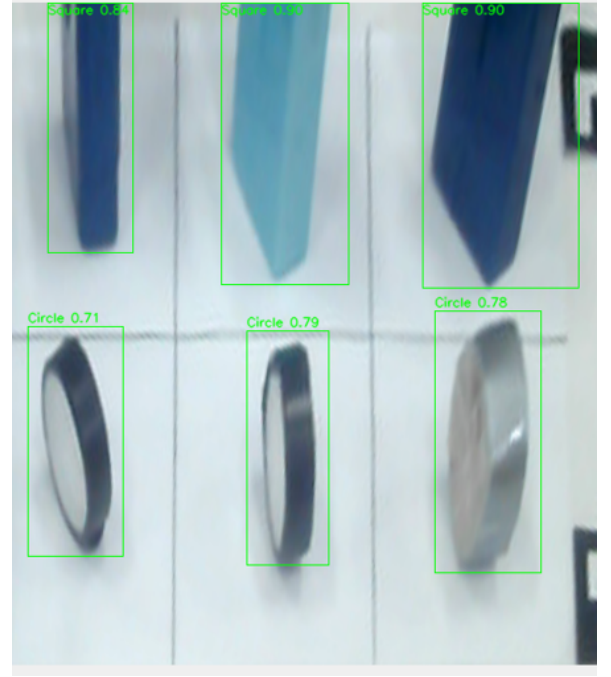


Fig. 16. Final physical-camera detection result after Stage 3. Both Square and Circle objects are detected.

TABLE I
THREE-STAGE TRAINING PROCEDURE

Stage	Domain	Initialization	Main Purpose
1	Phone images	YOLOv8n	Initial object learning
2	Gazebo simulation	Stage-1 weights	Simulation adaptation
3	Real robot camera	Stage-2 weights	Physical-view adaptation

The final result is shown in Fig. 16.

The previously missed Circle objects became detectable, while Square recognition remained stable.

This result confirmed that the primary cause of the failure was the mismatch between the original training viewpoint and the physical deployment viewpoint.

VII. EXPERIMENTAL RESULTS

A. Three-Stage Training Summary

The complete model-development sequence is summarized in Table I.

B. Quantitative Results

The initial and final quantitative detection results are listed in Table II.

After Stage 3, Circle achieved

$$P = 0.999, \quad R = 0.989,$$

$$\text{mAP50} = 0.995, \quad \text{mAP50-95} = 0.832.$$

Square achieved

$$P = 0.982, \quad R = 0.979,$$

$$\text{mAP50} = 0.981, \quad \text{mAP50-95} = 0.878.$$

The combined final results were

$$P = 0.991, \quad R = 0.984,$$

TABLE II
QUANTITATIVE DETECTION RESULTS

Model	Class	Precision	Recall	mAP50	mAP50-95
Initial	Circle	1.000	0.975	0.994	0.838
Initial	Square	0.991	0.982	0.988	0.872
Initial	All	0.996	0.978	0.991	0.855
Final	Circle	0.999	0.989	0.995	0.832
Final	Square	0.982	0.979	0.981	0.878
Final	All	0.991	0.984	0.988	0.855

TABLE III
MAJOR ENGINEERING PROBLEMS AND SOLUTIONS

Problem	Root Cause	Solution
<code>_ARRAY_API</code> error	NumPy 2.x incompatible with ROS 2 bindings compiled against NumPy 1.x	Pin NumPy to the 1.x series and verify <code>cv_bridge/OpenCV</code>
Class-ID mismatch	Training used 0=Circle and 1=Square, while the team interface required the reverse order	Map the class ID immediately before ROS publication
Camera overexposure	USB-camera brightness was incorrectly set to 10	Reset brightness to 0 using <code>v4l2-ctl</code>
Circle missed on robot	Strong viewpoint and perspective domain gap	Collect 48 real-camera images and perform transfer fine-tuning

$$\text{mAP50} = 0.988, \quad \text{mAP50-95} = 0.855.$$

Although the aggregate mAP values changed only slightly, the physical-camera performance improved substantially.

The final validation set contained 143 images, but only 8 images belonged to the newly introduced physical-camera validation subset.

Consequently, the aggregate validation metrics dilute the improvement specifically associated with the physical deployment domain.

For this reason, the direct physical retest provides important complementary evidence of the effectiveness of the Stage 3 adaptation.

VIII. ENGINEERING PROBLEMS AND SOLUTIONS

The major engineering problems encountered during the experiment are summarized in Table III.

IX. ROS 2 INTERFACE DELIVERED TO PART 2

The final perception interface is summarized in Table IV.

Publishing an empty `Detection2DArray` when no objects are detected is an important interface decision.

It ensures that the downstream state machine continues to receive explicit frame-level information instead of interpreting the absence of a message as a communication failure.

The interface also isolates Part 2 from the detector’s internal label ordering, Ultralytics implementation details, and visualization logic.

TABLE IV
ROS 2 INTERFACE DEFINITION

Item	Definition
Image input topic	<code>/image_raw</code>
Image message type	<code>sensor_msgs/Image</code>
Detection topic	<code>/detections</code>
Detection type	<code>vision_msgs/Detection2DArray</code>
External Class 0	Square
External Class 1	Circle
Bounding box	Pixel center + width/height
Confidence threshold	0.5 by default; configurable
No-object frame	Publish an empty detection array
Measured frequency	Approximately 30 Hz

X. DISCUSSION

A. Importance of Camera Viewpoint

The most important lesson from the experiment is that the training camera geometry should resemble the final deployment camera geometry as closely as possible.

A detector can achieve excellent aggregate validation metrics while still failing on a particular object class when the apparent shape of that object changes strongly with viewpoint.

In this experiment, the Circle class was considerably more perspective-sensitive than the Square class.

The tape rolls changed from approximately circular rings to compressed arc-like shapes under the physical camera angle, whereas the rectangular Square objects retained more stable visual geometry.

B. Value of the Simulation Training Stage

The simulation images were not merely temporary screenshots used to verify software execution.

They represented an intermediate visual domain between the original phone photographs and the fully integrated robotic system.

Explicitly introducing a Stage 2 simulation fine-tuning step made it possible to verify and adapt the detector inside Gazebo before moving to the physical platform.

Therefore, the complete experiment should be interpreted as progressive domain adaptation rather than a single model training followed by deployment.

C. Failure Diagnosis Before Retraining

Another important lesson was to diagnose each failure before deciding to retrain the network.

For the physical Circle detection problem, reducing the confidence threshold helped determine that the failure was not simply caused by score calibration.

For the overexposed image, checking the camera parameters showed that the failure was unrelated to YOLO.

Similarly, the class-ID inconsistency was an interface problem rather than a recognition problem and was therefore solved by deterministic mapping rather than dataset relabeling.

These targeted debugging steps preserved working components and reduced unnecessary retraining and annotation work.

D. Embedded Software Compatibility

The NumPy compatibility issue also demonstrates that embedded deployment requires careful dependency management.

A successful `pip` installation does not guarantee that previously compiled ROS 2 Python extensions remain compatible.

Changes to fundamental numerical libraries such as NumPy should therefore be followed by focused import tests for `cv_bridge`, OpenCV, and other binary Python extensions before the full robotic system is launched.

XI. CONCLUSION

A complete visual perception subsystem was developed for the desktop object sorting task.

The detector was developed through three distinct training stages.

Stage 1 used 676 manually collected mobile-phone images to learn the initial Square and Circle object representation.

Stage 2 introduced labeled Gazebo images and performed simulation-domain fine-tuning so that the detector could operate reliably in the integrated simulation environment.

Stage 3 introduced 48 additional images collected directly from the physical robot camera and addressed the strong viewpoint-induced domain gap affecting the Circle class.

In addition to model training, the experiment required the development of a complete ROS 2 detection pipeline and the resolution of several practical deployment problems, including class-ID inconsistency, NumPy/`cv_bridge` compatibility, camera overexposure, and real-camera distribution shift.

The final detector successfully recognizes both Square and Circle objects in the physical sorting environment.

It publishes standardized `vision_msgs/Detection2DArray` messages using the team-level class convention and operates at approximately 30 Hz on the Jetson Orin NX.

The experiment demonstrates that robust robotic perception depends not only on achieving high offline validation accuracy, but also on progressive domain adaptation, deployment-aware data collection, stable software dependencies, explicit module interfaces, and systematic failure diagnosis.

REFERENCES

- [1] Ultralytics, “Ultralytics YOLO Documentation.”
- [2] Open Robotics, “ROS 2 Humble Documentation.”
- [3] ROS 2, “`vision_msgs` `Detection2D` and `Detection2DArray` Message Definitions.”
- [4] OpenCV, “Open Source Computer Vision Library.”